

データ科学 レポート 3

神野 友哉

2022311042

愛知県立大学 情報科学部 情報科学科

2023 年 12 月 17 日

目 次

1	はじめに	2
2	課題 1	2
2.1	課題 1-1	4
2.1.1	目的	4
2.1.2	考察	4
2.1.3	結論	5
2.2	課題 1-2	5
2.2.1	目的	5
2.2.2	考察	5
2.2.3	結論	6
3	課題 2	6
3.1	課題 2-1	6
3.1.1	目的	6
3.1.2	考察	7
3.1.3	結論	7
3.2	課題 2-2	7
3.2.1	目的	7
3.2.2	考察	8
3.2.3	結論	9

1 はじめに

この実験では、MapReduce 法によって、メモリに収まらない程大きいデータが扱えるようになることを学ぶ。更に、実際にビッグデータを処理することで、活用方法も理解する。

2 課題 1

作業 1 ～ 2 を行う。処理時間計測の際、必ず実行二回目を記録する。

[作業 1] program_m1.m、map 関数、reduce 関数、mapimage.m を作成して MapReduce の逐次処理を行い、処理時間を計測する

[作業 2] program_m2.m を作成して MapReduce の並列処理を、2、4、6、8、12、16 並列で行い、処理時間を計測する。

作業 1 の program_m1.m はプログラム 1、作業 2 の program_m2.m はプログラム 2、map 関数、reduce 関数、mapimage.m は、それぞれプログラム 3、4、5 である。

source 1: programm_1.m (逐次処理)

```
clear;
addpath('/cis/public/datasci/jikken/L8data');
inMatlab=mapreducer(0);

tdir='/cis/public/datasci/class/taxi-yellow/2017/';
path1={['tdir 'yellow_tripdata_2017-01.csv'],...
        ['tdir 'yellow_tripdata_2017-02.csv']];

tic
ds=datastore(path1);
ds.SelectedVariableNames={'PULocationID', 'total_amount'};

ticBytes(gcp)
output=mapreduce(ds,@statmapper,@statreducer,inMatlab);
tocBytes(gcp)

r=readall(output); % Read output data
id=1:265; % Define number of zone
a=[id;r.Value{:}.fare]; % Create array of ID and fare
tmp=r.Value{:}.num<100; % Identify index number for (data size) < 100
fare_ave=a(2,:);
fare_ave(tmp)=nan; % Remove data for (data size) < 100
delete result*
elpt=toc
```

source 2: programm_2.m (並列処理)

```
clear;
addpath('/cis/public/datasci/jikken/L8data');
p=parpool(15);
inPool=mapreducer(p);

tdir='/cis/public/datasci/class/taxi-yellow/2017/';
path1={['tdir 'yellow_tripdata_2017-01.csv'],...
        ['tdir 'yellow_tripdata_2017-02.csv']];

tic
ds=datastore(path1);
ds.SelectedVariableNames={'PULocationID', 'total_amount'};

ticBytes(gcp)
output=mapreduce(ds,@statmapper,@statreducer,inPool);
tocBytes(gcp)
```

```

r=readall(output); % Read output data
id=1:265; % Define number of zone
a=[id;r.Value{:}.fare]; % Create array of ID and fare
tmp=r.Value{:}.num<100; % Identify index number for (data size) < 100
fare_ave=a(2,:);
fare_ave(tmp)=nan; % Remove data for (data size) < 100
delete result*
elpt=toc

```

source 3: map 関数

```

function statmapper(data,~,intermKVStore)

fare=data.total_amount;
puid=data.PULocationID;

tmpid=fare>300|fare<=0; %Identify index number for anomaly
fare(tmpid)=[]; % Remove data for anomaly
puid(tmpid)=[]; % Remove data for anomaly

for z=1:265
    tmp=puid==z;
    tmp2=fare(tmp);
    fare_sum(z)=sum(tmp2); % Calculate sum of fare for each zone
    fare_num(z)=size(tmp2,1); % Calculate number of sample for each zone
end
% Save results of map function that is subsequently read in reduce function
add(intermedKVStore,'key',{[fare_sum' fare_num']});

```

source 4: reduce 関数

```

function statreducer(~,intermValIter,outKVStore)

tmpfare=[];tmpnum=[];
c=0;
while hasnext(intermedValIter) % Check if results from map function still exist
    tmp=getnext(intermedValIter); % Get results from map function
    tmp2=tmp{:}; % Cell array to double array
    c=c+1;
    tmpfare(c,:)=tmp2(:,1);
    tmpnum(c,:)=tmp2(:,2);
end

stat.fare=sum(tmpfare,1)/sum(tmpnum,1); % Calculate averaged fare of each zone
stat.num=sum(tmpnum,1); % Calculate total number of each zone

add(outKVStore,'count',stat); % Save output
end

```

source 5: mapimage.m

```

function mapimage(param,id)

lw=1;
load colorv
load mat4im

input=[id' param'/(max(param))];

tmp2=isnan(input(:,2));
input(tmp2,:)=[];

figure(2);
mapshow(rlon_mat,rlat_mat,rJ);hold on
S=shaperead('Export_Output');
for z=1:size(S,1)
    mapshow(S(z),'facecolor','none','edgecolor','k',...
        'linewidth',lw);hold on;
end
hold on;
paint_shape(input,S,lw,c);
set(gca,'fontsize',10);colormap(jet);colorbar;
xlabel('Longitude');ylabel('Latitude');

```

```
hold off;
```

2.1 課題 1-1

作業 1, 2 の結果を利用して、コア数毎 ($N \geq 2$) に、計算時間 (t_n)、速度向上度 (S_n)、Karp-Flatt metric (K_f) を計算し、結果を折れ線グラフ (横軸は並列数) でプロットせよ

2.1.1 目的

MapReduce 法の並列処理の効果を確かめる。

2.1.2 考察

プログラム 1 を実行し、その処理時間が逐次処理時間となる。その後、Parpool(N) で設定したワーカー数でプログラム 1 と同様の処理を並列で行うプログラム 2 を実行し、並列処理時間のデータを得る。逐次処理時間と各並列処理時間を比較する。結果は表 1 になった。表 1 を見ればすぐに判ることだが、並列処理の

表 1: 処理時間の観測結果

N	1 (逐次処理)	2	3	4	5	6	7	8
$t_n(s)$	48.7170 (t_s)	22.5342	17.2881	15.9825	11.5062	10.0785	9.0965	8.3843
N	9	10	11	12	13	14	15	16
$t_n(s)$	7.6002	7.1209	7.0618	6.9421	6.2595	6.1114	6.0952	6.2074

コア数 N が増加するにつれて、処理時間は ($N = 16$ を除いて) 単調減少である。そのため、MapReduce 法の並列処理によって高速化されていると考えてよい。本題に入るが、得られた処理時間のデータから、速度向上度 ($S_n = \frac{t_s}{t_n}$, t_s は逐次処理時間 t_1) を計算したものが表 2 である。更に、プログラム 6 を実行するこ

表 2: 速度向上度 (S_n) の計算結果

N	1 (逐次処理)	2	3	4	5	6	7	8
S_n	1	0.4626	0.3549	0.3281	0.2362	0.2069	0.2069	0.1867
N	9	10	11	12	13	14	15	16
S_n	0.1560	0.1462	0.1450	0.1450	0.1285	0.1254	0.1251	0.1274

とにより、速度向上度 (S_n) から Karp-Flatt metric (K_f) を計算し、 N に対してプロットしたものが、図 1 である。図 1 から、 $N = 14$ までは K_f は増加傾向にあると考えられるが、 $N = 14$ から減少しているため、 $N = 14$ 周辺が取りえる最大となるのではないか。

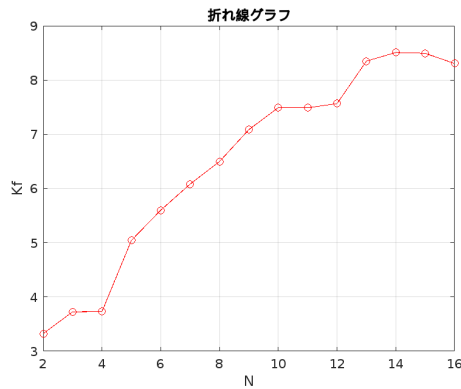


図 1: N と K_f の関係

source 6: 計算とプロット

```
clear;
N = 1:16;
tn = 48.7170;
ts = [48.7170 22.5342 17.2881 15.9825 11.5062 10.0785 9.0965
      8.3843 7.6002 7.1209 7.0618 6.9421 6.2595 6.1114 6.0952 6.2074];
sn = ts./tn;
kf = (N./sn - 1)./(N - 1);

plot(N, kf, 'c-o');
xlabel('N'), ylabel('Kf');
title('折れ線グラフ');
grid on;
print('wk2-1.png', '-dpng');
```

2.1.3 結論

この実験で用いたプログラムの処理では、MapReduce 法による効率化が確認された。しかし、並列処理のコア数 N を無尽蔵に増やす必要はないと結論づけてよいだろう。

2.2 課題 1-2

MapReduce のアルゴリズムの利点・欠点について調査せよ

2.2.1 目的

MapReduce 法の使うべき場合とそうでない場合を見極めるには、そのアルゴリズムの利点・欠点を知る必要があるため。

2.2.2 考察

MapReduce 法の利点として、巨大なデータを分割してメモリに読み込み、計算を実施するため、メモリの上限を上回るような巨大なデータを扱えるようになる。欠点として、データがメモリに収まるような場合は、効率化が見込めない。また、巨大なデータを構成する微細なデータ同士が複雑に関係する場合も非効率な場合もある。

2.2.3 結論

MapReduce 法は、メモリよりも大きいデータ (ビッグデータ) で活用するべきである。

3 課題 2

16 並列による MapReduce 処理を利用し、ニューヨークのイエローキャブ利用のビッグデータから、料金や走行距離、昇降地点等の情報を可視化する。それによって得られた図から、データの分析を行う。

3.1 課題 2-1

プログラム 2 をコピーして、読み込み対象のデータを 2017 年 1 月から 12 月までとなるよう修正したプログラム 7 を作成せよ。その後、処理結果 (fare_ave) をプログラム 5 により可視化して得られる図を示せ。また、その図と地域の土地利用に関する特徴を調べて見比べ、今回の結果が得られた理由を考察せよ。ただし、土地利用に関する特徴は、GoogleMap 等のウェブ上で閲覧可能な地図アプリケーション等を利用してよい。

source 7: 料金 (1 年分 16 並列)

```
clear;
addpath('/cis/public/datasci/jikken/L8data');
p=parpool(16);
inPool=mapreducer(p);

tdir='/cis/public/datasci/class/taxi-yellow/2017/';
path1={tdir 'yellow_tripdata_2017-01.csv'},...
    {tdir 'yellow_tripdata_2017-02.csv'},...
    {tdir 'yellow_tripdata_2017-03.csv'},...
    {tdir 'yellow_tripdata_2017-04.csv'},...
    {tdir 'yellow_tripdata_2017-05.csv'},...
    {tdir 'yellow_tripdata_2017-06.csv'},...
    {tdir 'yellow_tripdata_2017-07.csv'},...
    {tdir 'yellow_tripdata_2017-08.csv'},...
    {tdir 'yellow_tripdata_2017-09.csv'},...
    {tdir 'yellow_tripdata_2017-10.csv'},...
    {tdir 'yellow_tripdata_2017-11.csv'},...
    {tdir 'yellow_tripdata_2017-12.csv'}};

tic
ds=datastore(path1);
ds.SelectedVariableNames={'PULocationID', 'total_amount'};

ticBytes(gcp)
output=mapreduce(ds,@statmapper,@statreducer,inPool);
tocBytes(gcp)

r=readall(output); % Read output data
id=1:265; % Define number of zone
a=[id;r.Value{:}.fare]; % Create array of ID and fare
tmp=r.Value{:}.num<100; % Identify index number for (data size) < 100
fare_ave=a(2,:);
fare_ave(tmp)=nan; % Remove data for (data size) < 100
delete result*
elpt=toc
```

3.1.1 目的

ニューヨークのイエローキャブの利用料金と都市の土地利用との関連を調べる。

3.1.2 考察

図 2a は、プログラム 7 を実行したのち、コマンドラインで `mapimage(fare_ave,id)` とタイプして実行した結果である。また、図 2b は、ニューヨークの最も有力な交通として一日 360 万人に利用されるニューヨーク地下鉄である。これは、ニューヨークの中心部マンハッタンとブルックリンに多くの路線が通っている。



図 2: ニューヨークのイエローキャブ利用についての資料

図 2 から見て取れるのは、地下鉄の路線の密集度合いがイエローキャブの利用料金に負の相関関係があるということである。その原因を考察する。ニューヨーク中心部における地下鉄の利用料金が一律 2.9 ドルと、イエローキャブのそれに対し価格優位であることや、公共交通機関の定時制 (日本のそれと比べていい加減な面も否定できないが) が挙げられるのではないかと考えられる。国土交通省の昭和 62 年度運輸白書によると、都市部では通勤・通学の際に公共交通機関が多く利用されるという [3]。ここからも、日本の国家機関のデータではあるが、今回の実験の結果の理由として援用できるのではないかと考えられる。

3.1.3 結論

ニューヨーク中心部も日本の都会のそれと同様に、公共交通機関が利用される傾向にあると考えられる。

3.2 課題 2-2

2017 年 1 月から 12 月までのデータを対象に、`trip_distance`(走行距離) のデータを用いて単位距離当たりの料金 (料金の総和を距離の総和で除したもの) を `PULocationID` ごとに求め、プログラム 5 により可視化せよ。また、可視化した結果と地域の土地利用に関する特徴を調べ、今回の結果が得られた理由を考察せよ。ただし、支払料金が 0 ドル以下または 300 ドルより大きい、または、走行距離が 0.5 km 以下または 100 km より大きいものは除外すること。

3.2.1 目的

料金や走行距離のはずれ値を除外したデータを用いることで、より実態に沿った考察をしやすくするため。

3.2.2 考察

課題 2-2 のために走行距離のデータを処理できるよう、プログラム 7、3、4 からそれぞれプログラム 8、9、10 を作成した。

source 8: 単位距離当たりの料金 (1 年分 16 並列)

```
clear;
addpath('/cis/public/datasci/jikken/L8data');
p=parpool(16);
inPool=mapreducer(p);

tdir='/cis/public/datasci/class/taxi-yellow/2017/';
path1={tdir 'yellow_tripdata_2017-01.csv'},...
      {tdir 'yellow_tripdata_2017-02.csv'},...
      {tdir 'yellow_tripdata_2017-03.csv'},...
      {tdir 'yellow_tripdata_2017-04.csv'},...
      {tdir 'yellow_tripdata_2017-05.csv'},...
      {tdir 'yellow_tripdata_2017-06.csv'},...
      {tdir 'yellow_tripdata_2017-07.csv'},...
      {tdir 'yellow_tripdata_2017-08.csv'},...
      {tdir 'yellow_tripdata_2017-09.csv'},...
      {tdir 'yellow_tripdata_2017-10.csv'},...
      {tdir 'yellow_tripdata_2017-11.csv'},...
      {tdir 'yellow_tripdata_2017-12.csv'};

tic
ds=datastore(path1);
ds.SelectedVariableNames={'PULocationID', 'total_amount', 'trip_distance'};

ticBytes(gcp)
output=mapreduce(ds,@statmapper2,@statreducer2,inPool);
tocBytes(gcp)

r=readall(output); % Read output data
id=1:265; % Define number of zone
a=[id;r.Value{:}.fareptrdi]; % Create array of ID and fare
tmp=r.Value{:}.num<100; % Identify index number for (data size) < 100
fare_ave=a(2,:);
fare_ave(tmp)=nan; % Remove data for (data size) < 100
delete result*
elpt=toc
```

source 9: はずれ値を除いた map 関数

```
function statmapper2(data,~,intermKVStore)

fare=data.total_amount;
puid=data.PULocationID;
trdi=data.trip_distance;

tmpid=fare>300|fare<=0|trdi<=0.5|trdi>100; %Identify index number for anomaly
fare(tmpid)=[]; % Remove data for anomaly
puid(tmpid)=[]; % Remove data for anomaly
trdi(tmpid)=[];

for z=1:265
    tmp=puid==z;
    tmp2=fare(tmp);
    tmp3=trdi(tmp);
    fare_sum(z)=sum(tmp2); % Calculate sum of fare for each zone
    fare_num(z)=size(tmp2,1); % Calculate number of sample for each zone
    trdi_sum(z)=sum(tmp3);
end
% Save results of map function that is subsequently read in reduce function
add(intermedKVStore,'key',{[fare_sum' fare_num' trdi_sum]});
```

source 10: はずれ値を除いた reduce 関数

```
function statreducer2(~,intermValIter,outKVStore)

tmpfare=[];tmpnum=[];tmptrdi=[];
```



```

c=0;
while hasnext(intermValIter) % Check if results from map function still exist
    tmp=getnext(intermValIter); % Get results from map function
    tmp2=tmp{:}; % Cell array to double array
    c=c+1;
    tmpfare(c,:)=tmp2(:,1);
    tmpnum(c,:)=tmp2(:,2);
    tmptrdi(c,:)=tmp2(:,3);
end

stat.farepertrdi=sum(tmpfare,1)./sum(tmptrdi, 1); % Calculate fare per distance of each zone
stat.num=sum(tmpnum,1); % Calculate total number of each zone

add(outKVStore, 'count', stat); % Save output
end

```

図3は、プログラム8を実行したのち、コマンドラインで `mapimage(fare_ave,id)` とタイプして実行した結果である。

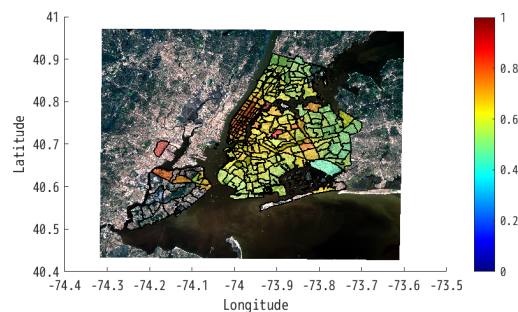


図3: ニューヨークの土地とイエローキャブの単位距離当たりの料金 (1年)

図3はまるで図2aの色相を逆にした様であった。この原因を考察する。ニューヨークの中心部マンハッタンの単位距離当たりの料金が高かった原因は経済の中心であるため、利用する人が多く存在することが考えられる。また、憶測でしかないが、ニューヨーク州のイエローキャブが初乗り 2.5 ドルで 1.5 マイル (約 320m) おきに 0.5 ドルずつ増加するため、少しでも安く済ませようとメーターが増加しないギリギリで降り、目的地までの残りを徒歩で歩く人がそこそこの割合でいるのではないか。

3.2.3 結論

ニューヨークの中心部マンハッタンでの客単価が高いことが判明した。

参考文献

- [1] MathWorks: "MapReduce 入門", MathWorks. 2023, https://jp.mathworks.com/help/matlab/import_export/getting-started-with-mapreduce.html, (参照 2023-12-17).
- [2] 世界お値段調査隊: "NY 地下鉄初乗り 420 円 幻じゃない!?改札の「ゴースト」", 日経新聞. 2023, <https://www.nikkei.com/article/DGXZQOGN2602Y0W3A820C2000000/#:~:text=1%E6%97%A5%E5%BD%93%E3%81%9F%E3%82%8A%E3%81%AE%E4%B9%97%E5%AE%A2%E6%95%B0,%E3%81%99%E3%82%8B%E8%>

A6%B3%E5%85%89%E5%9C%B0%E3%81%A7%E3%82%82%E3%81%82%E3%82%8A%E3%81%BE%E3%81%99%E3%80%82, (参照 2023-12-17).

- [3] 国土交通省: ”1 都市交通の整備”, 国土交通省. 1987,
<https://www.mlit.go.jp/hakusyo/transport/shouwa62/ind000502/001.html>, (参照 2023-12-17).